A3-PS012-TC045

Authorised DNS Resolver Exposure Assessment

Aditya V J
aditya.vj@btech.christuniversity.in

Dinesh Babu R S
dinesh.babu@btech.christuniversity.in

Dr. Gokulapriya R
r.gokulapriya@christuniversity.in

AIORI Hackathon 3

October 13, 2026

Problem Description

Open recursive DNS resolvers enable DDoS amplification and must be assessed safely.

- Open recursive resolvers answer anyone, so attackers use them as reflectors in DDoS amplification attacks.
- Operators need a precise, repeatable way to test whether a server offers recursion to an external client.
- A single flag such as RA is weak evidence, and a timeout is ambiguous.
- Testing must stay ethical: authorised lab only, up to 20 servers, no spoofing, no Internet-wide scanning, bounded rate.

Abstract

ResolverLens is a probe tool that checks whether up to 20 authorised lab DNS servers provide recursion to an external client, and labels each one Recursive, Authoritative-only, Refused or Inconclusive. It confirms recursion using our own canary server, combines flags, answers and canary evidence, limits its query rate with token buckets, and scores every verdict against expected labels.

Background

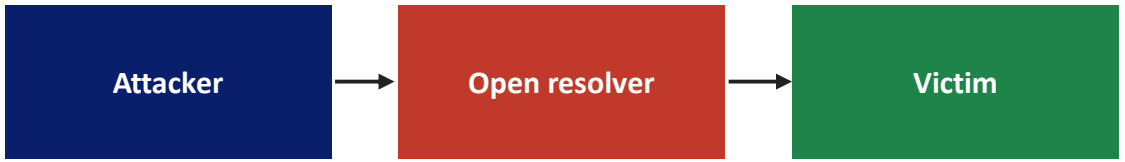
Recursion, how it is abused, and what RFC 5358 asks operators to do.

Normal recursion



A recursive resolver finds answers for clients by asking other servers, then caches them. An open resolver does this for anyone on the Internet, often through a default or careless configuration [1].

Reflection attack



UDP has no handshake, so a sender address can be forged and replies land on the victim. Open resolvers are widely abused this way [1], [6].

RFC 5358 [1] (BCP 140) tells operators not to offer recursion to everyone, only to their intended clients. This is the policy we assess against.

What is an RFC? A Request for Comments is a free, numbered IETF document that defines an Internet protocol or practice. BCP means Best Current Practice. All are published at rfc-editor.org.

DNS Message Structure

Every query we send and every reply we read uses this layout [2].

Header: 12 bytes



Flags field: 16 bits



Blue: set by us (RD). **Red: read from the reply (AA, TC, RA, RCODE).**

Body: four sections

Question	**Answer**	**Authority**	**Additional**
name, type, class asked	records that answer it	who is authoritative, referrals	extra helpful records

Example Nmap's dns-recursion check sends one query with RD=1 for a third-party name and reads RA and RCODE in the reply [7].

Result codes: RFC 1035 [2]

We may see	What it suggests
Answer + RA=1	Server looked it up
Answer + AA=1	Own-zone data
Empty answer	No recursion (or referral)
REFUSED (5)	Policy denies us
SERVFAIL (2)	Lookup failed
NXDOMAIN (3)	Name does not exist
TC=1	Too big, retry over TCP
No reply	Loss or firewall drop

Understanding the Problem Statement

What PS-012 requires, and how we meet it.

PS-012 requirement	How we meet it
Assess up to 20 authorised servers	Hard allowlist of 20 lab IPs; anything else is blocked
Focus only on recursion exposure	Test only whether a target recurses for an outsider
Controlled zone + separate client	Our canary zone; external probe client on its own range
Four response classes	Recursive, Authoritative-only, Refused, Inconclusive (with reason)
Compare with seeded ground truth	Every lab server has a known expected label
Rate controls, bounded, reproducible	Token buckets, fixed seed
No scans, spoofing, amplification	Isolated lab, real source IPs, tiny queries
Out of scope	Cache snooping, DNSSEC, public servers

Expected outcome	Our deliverable
DNS resolver probe tool	ResolverLens probe engine
Controlled labelled test cases	20 seeded lab servers
Configurable rate control	Token buckets set in config.yaml
Classification + ground-truth comparison	Decision rules + confusion matrix
Exposure assessment report	Auto-generated per run
False/inconclusive analysis	Reason codes + root-cause notes
Final demo and report	Live run + written report

Literature & RFC Review

Standards, research and tools relevant to recursion exposure.

Ref	Source	Key idea	How we use it
[1]	**RFC 5358 (BCP 140)**	Do not offer recursion to everyone	Policy we assess against
[2]	**RFC 1035**	Message format, flags, result codes	Fields we send and read
[3]	**RFC 6891 (EDNS0)**	Larger UDP messages	We keep our buffer small
[4]	**RFC 7766 (DNS over TCP)**	Servers must support TCP	Retry over TCP when TC=1
[5]	**RFC 8906 (BCP 231)**	Silent or wrong replies are common	Why timeouts are Inconclusive
[6]	**Kührer et al., IMC 2015**	Classifies open DNS resolvers at scale	Shows the open-resolver risk
[7]	**Nmap dns-recursion**	One RD=1 query, checks RA and RCODE	Baseline we improve on

Gap we address: existing checks read flags from one query. We add canary evidence and a control client.

Proposed Solution: ResolverLens

A flag only says what a server claims; our canary server shows what it did.

The idea Ask the target for a made-up name only our canary server can answer. If the target looks the name up, the canary server logs it: that is our evidence of recursion.

Key terms (used on every slide)

Target server	A lab server we assess
Canary server	Our own DNS server for a test zone; logs every query
Canary query	Our probe for a unique made-up name in that zone
Canary evidence	That name appears in the canary log, so the target looked it up
External probe client	The outsider machine where our tool runs
Control client	Inside the allowed range; shows a server can recurse at all
Expected label	The known right answer (seeded ground truth) for each lab server

1 Build the lab
Docker lab: 20 targets, canary server, two clients, expected labels

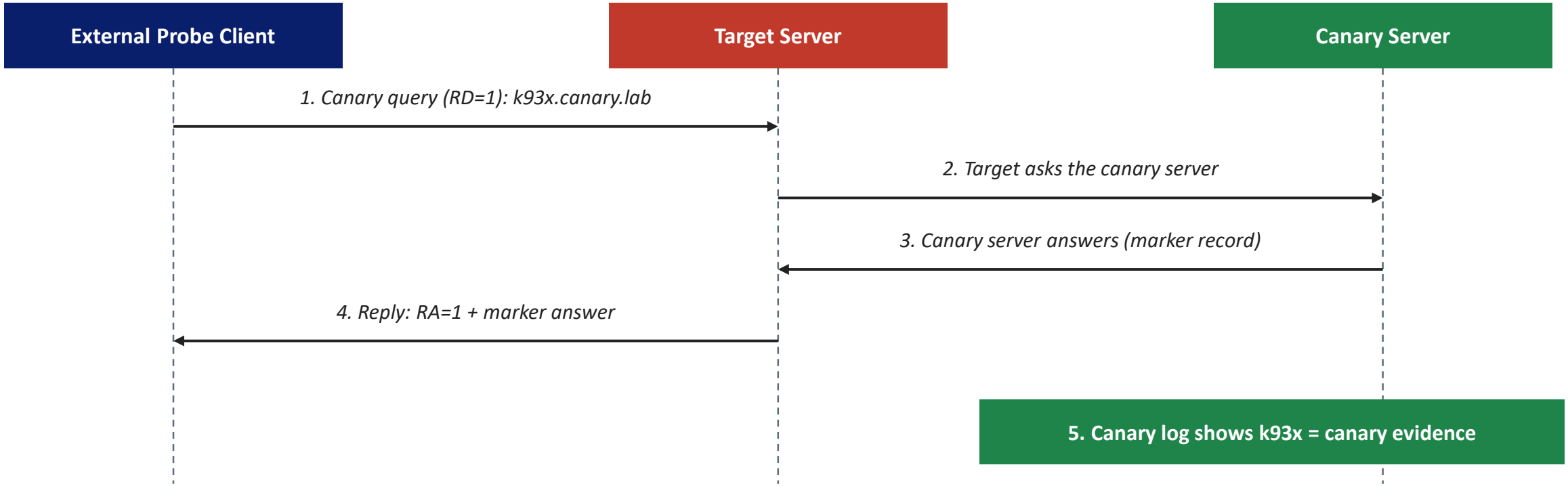
2 Probe each target
Canary queries from the external client, rate-limited; control client checks

3 Classify
Rules read flags, RCODE, answer and canary evidence

4 Score and report
Compare with expected labels; confusion matrix and report

How One Probe Works

One canary query, from the external client to the canary evidence.



Why a fresh name each time A baseline query first checks whether the target answers its own zone (AA=1). Each canary name is used once, so caching cannot hide the lookup. No canary evidence and no answer indicates no recursion.

Classification Logic

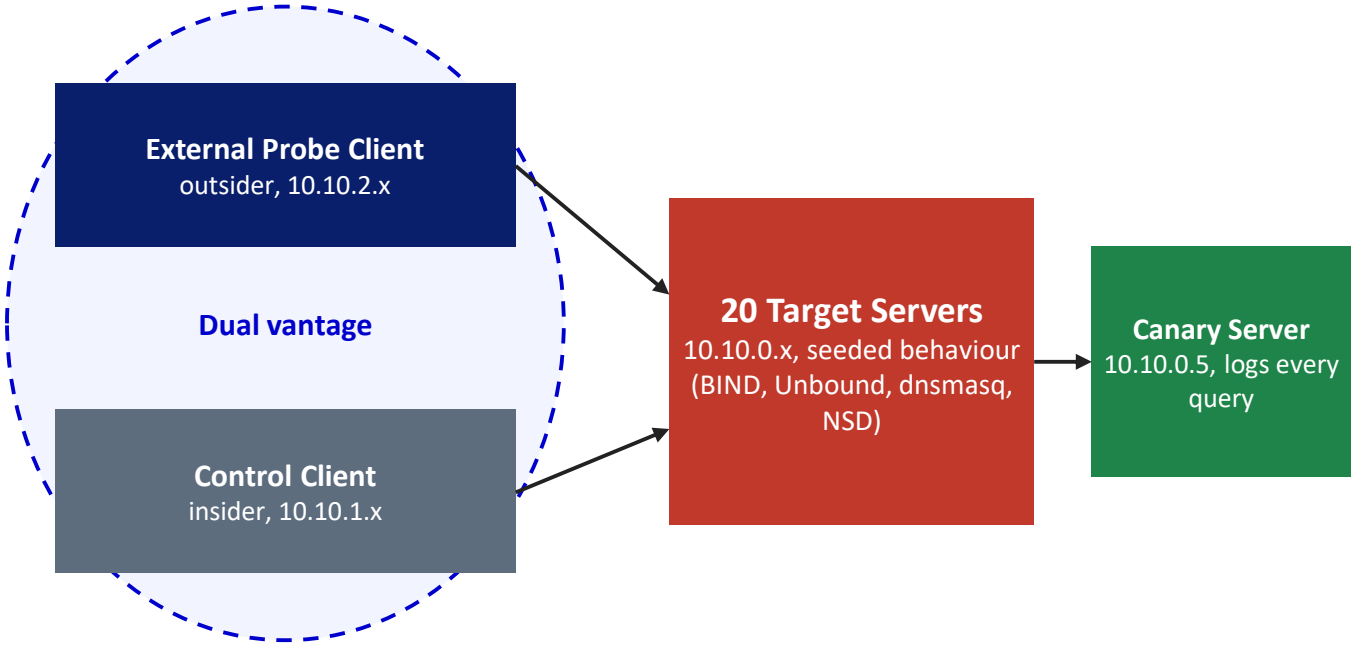
Rules are checked in order; the first match decides the class.

Class	Fields we read	Example reply seen
1 Recursive	RA=1, RCODE NOERROR, the marker answer, and canary evidence in the log	qr rd ra, NOERROR, answer = marker record; canary log shows the name
2 Refused	RCODE REFUSED for both the baseline and the canary query	status REFUSED, no answer, for every query from us
3 Authoritative-only	Baseline answered with AA=1; canary query gets no answer; no canary evidence	baseline: qr aa NOERROR; canary query: REFUSED or empty, RA not set
4 Inconclusive	Anything else, with a reason code: TIMEOUT, SERVFAIL, TC_FAIL, CONFLICT	No reply to 3 attempts, which cannot be told from packet loss [5]; or the 3 attempts disagree

Attempts and reasons: each target gets 3 canary attempts with fresh names; they must agree, otherwise CONFLICT. TC_FAIL means a truncated reply that TCP could not complete.

Test Lab and Expected Labels

A lab where the correct answer for every server is known in advance.



Isolated lab: no Internet access. A target that recurses sends an upstream query to the canary server.

Figure: Test lab topology and planned seeded mix (labels are fixed at build time).

Seeded type	n	Expected label
Open recursive	5	Recursive
Policy-refusing	4	Refused
Authoritative-only	4	Authoritative-only
Forwarders	3	Recursive
Heavy loss / rate-limited	2	Inconclusive
Broken / silent	2	Inconclusive

Why REFUSED? Policy-refusing servers allow only the control client's range: it gets service, we get REFUSED.

Lab Implementation

How we build the lab on one laptop with Docker, and which tools do what.

1 Describe
targets.yaml lists each server, address, software and expected label

2 Generate
a script writes the Docker Compose file and one config per server

3 Start
docker compose up creates about 23 containers on an isolated network

4 Check
a manual dig test and the tool's preflight check confirm the canary log works

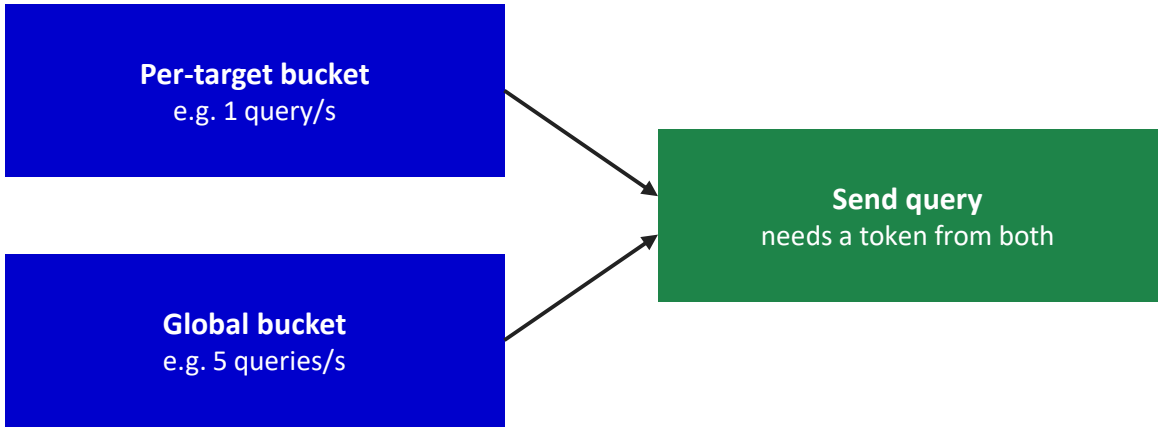
5 Run
the tool runs inside the external probe client container

Tool	Role
Docker Compose	Starts and connects all containers
BIND9, Unbound, dnsmasq	Targets and canary server
tc netem, firewall rule	Packet loss, delay, silent drops
Python + dnspython	Probe engine and classifier
pandas + Jinja2	Results tables and report page
Git + GitHub	Version control, mentor access

Target type	How we configure it
Open recursive	recursion on, answers any client
Policy-refusing	limited to 10.10.1.0/24
Authoritative-only	recursion off, one small zone
Forwarder	forwards all queries to an open target
Heavy loss / rate limit	tc netem loss; response rate limit
Broken / silent	firewall drop, or bad upstream

Rate Control and Safeguards

Test traffic stays within the lab and is rate-bounded.



Token bucket: a counter that refills at a fixed rate up to a maximum. Each query uses one token; when the bucket is empty the tool waits. Rate, burst, timeout, retries and seed are set in config.yaml. A dry-run mode prints the plan without sending anything.

Safeguards

- Allowlist checked before any packet (max 20)
- Real source address only: no spoofing
- Small A/TXT queries, no ANY queries
- Isolated lab, no Internet access
- Stops if reply size looks unusually large
- Never aimed at public resolvers
- Kill switch (Ctrl+C) and an audit log of every query

Monitoring and Risk Mitigation

How we watch each run, and what could go wrong.

Monitoring

- Preflight: canary logs a test query; control client gets recursion from a known open target
- Live console line per target, and an audit log of every query
- Stability check: run twice, flag servers whose verdict changes
- Replay: re-runs the classifier on saved evidence for mentor rechecks

Risks and mitigations

Risk	Mitigation
Cache hides the lookup	Fresh unique name for each attempt
Silent firewall drop	Control client + TIMEOUT reason
Forwarder chains	Match by canary name, not source IP
Rate-limited servers	Low rate, TCP retry
Canary log unreadable	Marker answer still proves resolution

Optimization Proposed

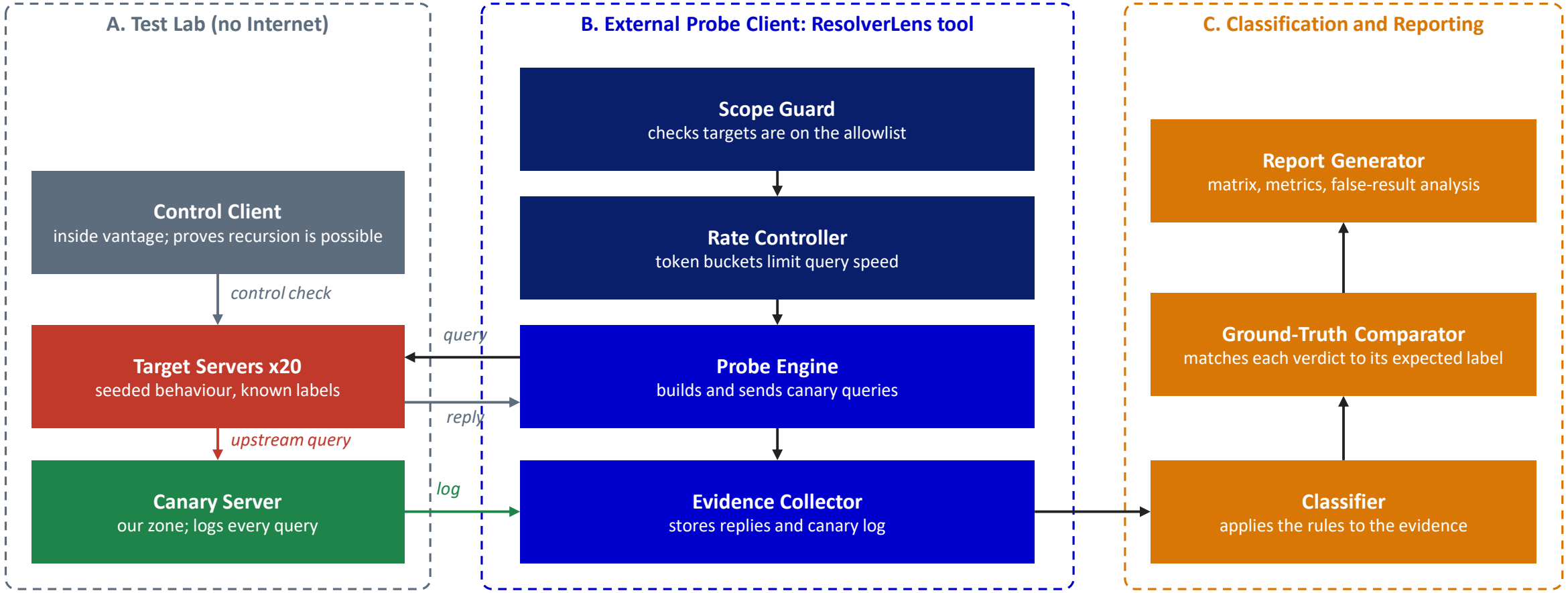
Proposed advantages over RA-flag-only checking.

Aspect	Before: single flag	After: ResolverLens	Requirement served
Evidence	One flag (RA)	Flags, RCODE, answer and canary evidence	Classification
Where recursion happens	Flag cannot show it	Canary log shows the target's own lookup	Recursive vs authoritative-only
Caching	Repeated name may be cached	Fresh unique name per attempt	Reproducibility
Packet loss	Timeout is ambiguous	3 attempts, TCP retry, reason code	Inconclusive analysis
Refusals	Policy vs fault unclear	Control client shows REFUSED is policy	Separate vantage
Rate control	Fixed delay	Token buckets, per target and global	Bounded testing
Repeat runs	One-off run	Seed, stability check, replay	Reproducibility
Scoring	None	Confusion matrix vs expected labels	Ground truth

Expected benefit: fewer false labels, explained inconclusive results, and replayable runs. To be validated against expected labels.

Solution Architecture & Design

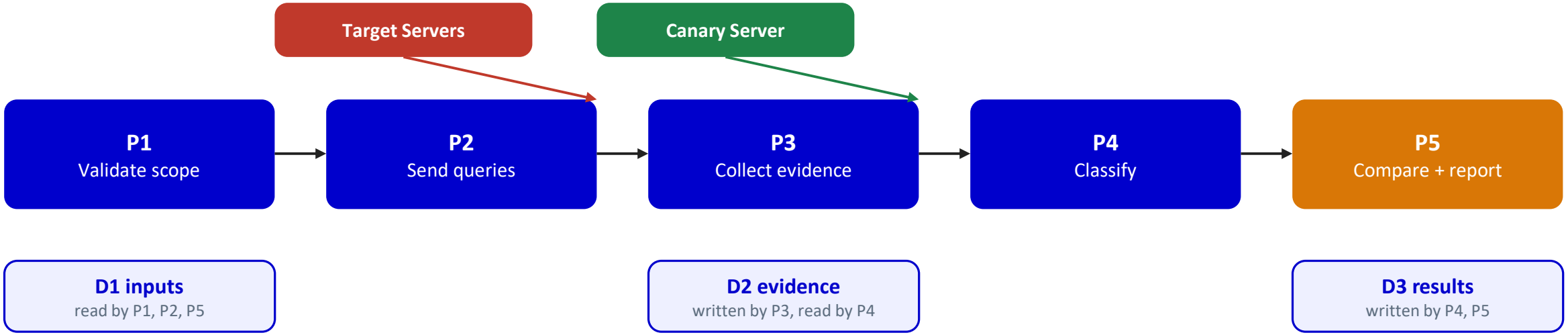
Three parts: the test lab, the ResolverLens tool, and classification and reporting.



The tool runs only on the external probe client. The control client is deliberately simple: it just sends a plain recursion query to prove a server can recurse at all.

Data Flow Diagram

How data moves from the input files to the final report.



Store	Files and contents	Written by	Read by
D1 inputs	targets.yaml: 20 IPs, software, expected label. config.yaml: rate, timeout, retries, seed, zone name	We write them when building the lab	P1, P2, P5
D2 evidence	evidence.jsonl: one record per probe (name, flags, RCODE, answer, delay, canary hit); canary log; optional packet capture	P3	P4, replay
D3 results	results.csv: per server verdict, reason, expected label, match; report (HTML and PDF)	P4, P5	Us and mentors

Evaluation and Exposure Report

Verdicts are scored against expected labels, with reason codes for inconclusive results.

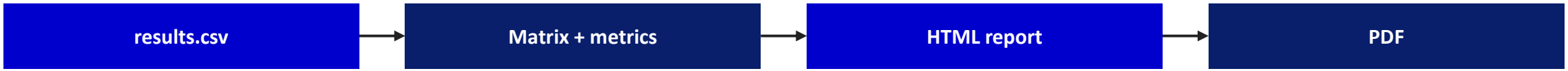
Expected \ Reported	Recursive	Authoritative-only	Refused	Inconclusive
Recursive	correct	error	error	abstain
Authoritative-only	error	correct	error	abstain
Refused	error	error	correct	abstain
Inconclusive	error	error	error	abstain

Illustration only: if 18 of 20 are decided and 17 are correct, accuracy is 17/18 and coverage is 18/20.

Each error is traced to a cause: cache, forwarder, ACL or packet loss.

Report contents

- Per-server verdict + evidence
- Precision and recall per class
- Accuracy on decided cases
- Coverage: share of servers decided
- Inconclusive count by reason
- False-result analysis
- Fix advice per RFC 5358 [1]



Deployment Timeline

Complete, working system live by 31 October.

1 Lab build

20 seeded targets, canary zone, ground-truth manifest

14 to 18 Oct

2 Probe engine

Scope guard, probes, token-bucket rate control

18 to 22 Oct

3 Classifier

Decision rules, reason codes, confusion matrix

22 to 25 Oct

4 Report

Stability check, replay, false/inconclusive analysis

25 to 29 Oct

5 Go live

Full run on all 20 targets, demo and final report ready

By 31 Oct

Methodology Iterative builds in the private team repository, with mentor oversight. Each change is checked against the seeded ground truth, and every run stores its seed and configuration for replay. Safety guardrails are active from the first test. The complete system is live and demo-ready by 31 October.

Bibliography

[1] J. Damas, F. Neves. Preventing Use of Recursive Nameservers in Reflector Attacks. RFC 5358 (BCP 140), 2008.

[2] P. Mockapetris. Domain Names: Implementation and Specification. RFC 1035 (STD 13), 1987.

[3] J. Damas, M. Graff, P. Vixie. Extension Mechanisms for DNS (EDNS(0)). RFC 6891 (STD 75), 2013.

[4] J. Dickinson et al. DNS Transport over TCP: Implementation Requirements. RFC 7766, 2016.

[5] M. Andrews, R. Bellis. A Common Operational Problem in DNS Servers: Failure to Communicate. RFC 8906 (BCP 231), 2020.

[6] M. Kührer, T. Hupperich, J. Bushart, C. Rossow, T. Holz. Going Wild: Large-Scale Classification of Open DNS Resolvers. ACM Internet Measurement Conference (IMC), 2015.

[7] Nmap Project. NSE script dns-recursion. nmap.org/nsedoc/scripts/dns-recursion.html

Thank You for Your Attention!

A3-PS012-TC045

Aditya V J • Dinesh Babu R S • Dr. Gokulapriya R

Authorised DNS Resolver Exposure Assessment • AIORI Hackathon 3